Data Catalogue

Retail Company — Synthetic Dataset

25 fichiers CSV · 6 fichiers JSON · Contenu en français · En-têtes en anglais

Vue d'ensemble des sources (31 tables)

Table / Fichier	Description	Format
`customers`	Clients enregistrés dans le CRM avec données personnelles, fidélité et canal d'acquisition	csv
`products`	Produits vendus — tarifs, attributs physiques, fournisseur, empreinte carbone	csv
`suppliers`	Fournisseurs liés aux produits, notés sur fiabilité, délai et contrat	csv
`transactions`	Lignes d'achat regroupées en tickets (un produit par ligne)	csv
`marketing`	Campagnes promotionnelles liées aux produits avec budget et canal	csv
`employees`	Données RH des employés assignés aux magasins, entrepôts ou siège	csv
`stores`	Magasins physiques avec localisation, surface et revenus annuels	csv
`warehouses`	Centres de distribution régionaux avec capacité et coût énergétique	csv
`store_warehouses`	Entrepôts arrière-boutique alimentant directement la salle de vente	csv
`hr_positions`	Référentiel des postes — compétences requises et contraintes horaires	csv
`logistics`	Trajets de transport entre entrepôts avec CO_2, délais et carburant	csv
`accounting`	Entrées comptables — loyer, énergie, salaires, taxe carbone, investissement	csv
`product_returns`	Retours produits par les clients avec motif et statut de remboursement	csv
`product_reviews`	Avis produits collectés en magasin ou en ligne (source distinguée)	csv
`store_maintenance`	Interventions de maintenance et rénovation dans les magasins	csv
`sales_targets`	Objectifs de vente par magasin et période avec réalisation effective	csv

Table / Fichier	Description	Format
`vehicle_types`	Référentiel des types de véhicules — capacité et émissions CO_2 par km	CSV
`carbon_regulations`	Prix du carbone par pays et par année (EU ETS ou taxe nationale)	CSV
`store_replenishment`	Flux de réapprovisionnement des entrepôts régionaux vers les magasins	CSV
`inventory`	Instantanés de stock par produit et par localisation à une date donnée	CSV
`supplier_orders`	Commandes d'achat envoyées aux fournisseurs avec délais et statuts	CSV
`price_history`	Historique des prix de vente et d'achat par produit et période	CSV
`employee_events`	Journal RH des événements par employé (embauche, promotion, départ...)	CSV
`economic_zones`	Indicateurs économiques et démographiques par pays et région	CSV
`budget_forecast`	Budgets prévisionnels vs réels par centre de coût et période fiscale	CSV
`web_sessions`	Sessions web clients — navigation, panier et événements d'achat	JSON
`customer_requests`	Demandes produits et avis magasin des clients (source : en ligne)	JSON
`marketing_notifications`	Historique des envois de notifications par campagne et par client	JSON
`stock_alerts`	État des alertes de stock par entrepôt avec historique de quantités	JSON
`employee_profiles`	Profils enrichis des employés — compétences, formations, évaluations	JSON
`product_catalog_web`	Catalogue web avec descriptions, tags, avis et historique de prix	JSON

Description détaillée des colonnes

Data Catalogue — Retail Company Synthetic Dataset

Column and table names: English **Business content (values, names, places):** French **Total sources:** 31 (25 CSV + 6 JSON)

Conventions

Symbol	Meaning
PK	Primary key
FK	Foreign key

Symbol	Meaning
NN	Not null
—	Nullable

CSV Tables

`customers`

Description: Customers registered in the CRM with personal data, loyalty tier and acquisition channel.

Column	Type	Null	Constraints	Description
customer_id	string	NN	PK, unique	Unique identifier — prefix `CUST_`
full_name	string	NN	non-empty	First and last name
email	string	NN	valid email format	Contact email
gender	string	NN	H / F / X	Gender
birth_date	date	NN	<= today, >= 1930-01-01	Date of birth
registration_date	date	NN	>= birth_date + 18 years	CRM registration date
loyalty_status	string	NN	bronze / argent / or / platine	Loyalty tier
country	string	NN	France / Allemagne / Espagne / Italie / Portugal	Country of residence
acquisition_source	string	NN	web / physique / campagne / parrainage	How the customer was acquired

No foreign keys.

`products`

Description: Products sold in stores — pricing, physical attributes, supplier link, carbon baseline.

Column	Type	Null	Constraints	Description
product_id	string	NN	PK, unique	Unique identifier — prefix `PROD_`
product_name	string	NN	non-empty	Commercial name
category	string	NN	Électronique / Alimentation / Vêtements / Maison / Hygiène	Main category
sub_category	string	—	—	Sub-category within category
is_fragile	boolean	NN	true / false	Requires fragile handling
volume_l	float	NN	> 0	Volume in litres

Column	Type	Null	Constraints	Description
weight_kg	float	NN	> 0	Weight in kilograms
selling_price	float	NN	>= 0	Unit selling price in EUR
purchase_cost	float	NN	>= 0	Unit purchase cost from supplier in EUR
margin_rate	float	NN	(selling_price - purchase_cost) / selling_price	Gross margin rate
storage_cost	float	NN	>= 0	Monthly storage cost per unit in EUR
launch_date	date	NN	<= today	Date the product was launched
expiry_date	date	NN	> launch_date	Expiry or end-of-life date
is_new_product	boolean	NN	true if (today - launch_date) < 180 days	New product flag
supplier_id	string	NN	FK → suppliers.supplier_id	Assigned supplier
manufacturing_origin	string	NN	non-empty	Country of manufacture
co2_per_kg	float	NN	>= 0	Base CO2 emissions in kg per kg of product (manufacturing footprint)

Foreign keys: - `supplier_id` → `suppliers.supplier_id`

`suppliers`

Description: Suppliers linked to products, rated on reliability, delivery, contract type and carbon modifier.

Column	Type	Null	Constraints	Description
supplier_id	string	NN	PK, unique	Unique identifier — prefix `SUPP_`
supplier_name	string	NN	non-empty	Company name
country	string	NN	France / Allemagne / Espagne / Italie / Portugal / Chine / États-Unis	Country of origin
reliability_score	float	NN	0.5 – 1.0	Delivery and quality reliability score
lead_time_days	integer	NN	> 0	Average lead time in days
contract_type	string	NN	exclusif / cadre / spot	Type of supply contract
co2_modifier_per_kg	float	NN	can be negative	CO2 added (or saved) per kg vs product baseline — negative = greener supplier
specialization	string	NN	one or more categories from products.category	Product categories supplied

Column	Type	Null	Constraints	Description
min_order_qty	integer	NN	> 0	Minimum order quantity in units

No foreign keys.

`transactions`

Description: Purchase lines grouped into tickets. Each row is one product line within a basket. Covers in-store and online sales.

Column	Type	Null	Constraints	Description
transaction_id	string	NN	PK, unique	Unique line identifier — prefix `TRX_`
ticket_id	string	NN	duplicated within a basket	Basket identifier — prefix `TICKET_`
customer_id	string	NN	FK → customers.customer_id	Purchasing customer
product_id	string	NN	FK → products.product_id	Purchased product
store_id	string	NN	FK → stores.store_id	Store where sale occurred
quantity	integer	NN	> 0	Number of units purchased
total_price	float	NN	>= 0	Total line price after discount in EUR
discount_applied	float	—	>= 0	Discount amount in EUR
customer_rating	float	—	0 – 10	In-store rating given by customer
purchase_date	date	NN	<= today	Date of purchase
purchase_time	time	NN	HH:MM	Time of purchase — enables temporal pattern analysis
sales_channel	string	NN	en ligne / en magasin / téléphone	Sales channel
payment_method	string	NN	carte / espèces / mobile	Payment method
session_id	string	—	FK → web_sessions.session_id — only when sales_channel = 'en ligne'	Web session that led to this purchase
expected_delivery_date	date	—	>= purchase_date — only when sales_channel = 'en ligne'	Promised delivery date
actual_delivery_date	date	—	>= purchase_date — only when sales_channel = 'en ligne'	Actual delivery date

Foreign keys: - `customer_id` → `customers.customer_id` - `product_id` → `products.product_id` - `store_id` → `stores.store_id` - `session_id` → `web_sessions.session_id` (nullable)

`marketing`

Description: Promotional campaigns linked to products. Multiple rows can share a `campaign_group_id` to form a single campaign across products.

Column	Type	Null	Constraints	Description
marketing_id	string	NN	PK, unique	Unique identifier — prefix `MKT_`
campaign_group_id	string	—	groups rows into one campaign	Campaign group identifier — prefix `CAMP_`
product_id	string	NN	FK → products.product_id	Target product
product_segment	string	—	Produits de niche / Produits grand public	Market segment
promotion_type	string	—	pourcentage_remise / remboursement_immediat / produit_gratuit	Type of promotion
discount_value	float	—	>= 0	Discount value (% or EUR depending on type)
target_customer_type	string	—	bronze / argent / or / platine	Target loyalty tier
promotion_date	date	—	—	Start date of promotion
duration_days	integer	—	> 0	Duration in days
channel	string	—	email / SMS / TV	Communication channel
budget	float	—	>= 0	Allocated budget in EUR
actual_cost	float	—	>= 0	Actual spend in EUR

Foreign keys: - `product_id` → `products.product_id`

`employees`

Description: HR data for employees assigned to stores, warehouses or headquarters.

Column	Type	Null	Constraints	Description
employee_id	string	NN	PK, unique	Unique identifier — prefix `EMP_`
full_name	string	NN	non-empty	Full name
position	string	NN	non-empty	Job title
department	string	NN	logistique / ventes / RH / informatique	Department
store_id	string	—	FK → stores.store_id	Assigned store (nullable)
warehouse_id	string	—	FK → warehouses.warehouse_id	Assigned warehouse (nullable)
hire_date	date	NN	<= today	Date of hire

Column	Type	Null	Constraints	Description
salary	float	NN	>= 0	Annual gross salary in EUR
age	integer	NN	15 – 80	Age in years
gender	string	NN	H / F	Gender
family_status	string	—	célibataire / marié(e) / divorcé(e) / veuf(ve)	Family situation
is_manager	boolean	NN	true / false	Manager flag
contract_type	string	NN	CDI / CDD / intérim	Employment contract type
contract_end_date	date	—	> hire_date — only for CDD / intérim	Contract end date
departure_reason	string	—	démission / licenciement / retraite / fin_contrat	Reason for leaving (null = still active)
education_level	string	—	bac / bac+2 / bac+3 / bac+5 / doctorat	Highest education level

Foreign keys: - `store_id` → `stores.store_id` - `warehouse_id` → `warehouses.warehouse_id`

`stores`

Description: Retail stores with geographic, real estate and financial characteristics.

Column	Type	Null	Constraints	Description
store_id	string	NN	PK, unique	Unique identifier — prefix `MAG_`
address	string	NN	non-empty	Street address
city	string	NN	non-empty	City
country	string	NN	France / Allemagne / Espagne / Italie / Portugal	Country
postal_code	string	NN	non-empty	Postal code
opening_date	date	NN	<= today	Store opening date
property_status	string	NN	loué / acheté	Real estate ownership status
surface_sqm	float	—	> 0	Store surface in m²
district_type	string	—	résidentiel / centre-ville / zone commerciale / industriel / rural / périphérie	District type
annual_revenue	float	—	>= 0	Annual revenue in EUR
renovation_done	boolean	NN	true / false	Whether renovation works were performed
renovation_cost	float	—	>= 0	Renovation cost in EUR
warehouse_id	string	NN	FK → warehouses.warehouse_id	

Column	Type	Null	Constraints	Description
				Main regional warehouse supplying this store

Foreign keys: - `warehouse_id` → `warehouses.warehouse_id`

`warehouses`

Description: Main regional distribution centres (DCs). Supply store_warehouses which in turn feed the sales floor.

Column	Type	Null	Constraints	Description
warehouse_id	string	NN	PK, unique	Unique identifier — prefix `ENT_`
city	string	NN	non-empty	City
country	string	NN	France / Allemagne / Espagne / Italie / Portugal	Country
surface_sqm	float	NN	> 0	Warehouse surface in m²
storage_capacity	float	NN	> 0	Maximum storage capacity in units
warehouse_type	string	NN	réfrigéré / sec / automatisé	Type of warehouse
monthly_energy_cost	float	NN	>= 0	Monthly energy cost in EUR
occupancy_rate	float	NN	0.0 – 1.0	Current stock fill rate

No foreign keys.

`store_warehouses`

Description: Store-level backroom warehouses. Receive replenishment from regional DCs and feed the sales floor directly.

Column	Type	Null	Constraints	Description
store_warehouse_id	string	NN	PK, unique	Unique identifier — prefix `SW_`
store_id	string	NN	FK → stores.store_id	Owning store
surface_sqm	float	NN	> 0	Surface in m²
storage_capacity	float	NN	> 0	Maximum storage capacity in units
current_stock_volume	float	NN	0 – storage_capacity	Current stock level in units

Foreign keys: - `store_id` → `stores.store_id`

`hr_positions`

Description: Reference table for job positions — schedule constraints, required skills, seniority level. One row per distinct position title.

Column	Type	Null	Constraints	Description
position	string	NN	PK, unique	Job title — matches employees.position
skills	string	NN	non-empty	Required skills, comma-separated
store_id	string	—	FK → stores.store_id	Applicable store (null = all)
warehouse_id	string	—	FK → warehouses.warehouse_id	Applicable warehouse (null = all)
min_schedule	string	NN	HH:MM	Earliest shift start time
max_schedule	string	NN	HH:MM	Latest shift end time
leave_days	integer	—	>= 0	Annual leave days
rotation	string	—	matin / après-midi / nuit / 3×8	Shift rotation type
seniority_level	string	—	junior / confirmé / senior / expert	Expected seniority level

Foreign keys: - `store_id` → `stores.store_id` - `warehouse_id` → `warehouses.warehouse_id`

`logistics`

Description: Transport trips between regional warehouses. Covers trucks of multiple sizes (including refrigerated) and trains. Enables CO2, delay and capacity analysis.

Column	Type	Null	Constraints	Description
trip_id	string	NN	PK, unique	Unique identifier — prefix `TRJ_`
vehicle_id	string	NN	non-empty	Specific vehicle identifier — prefix `VEH_`
vehicle_type_id	string	NN	FK → vehicle_types.vehicle_type_id	Vehicle type (size, mode, refrigerated)
transport_mode	string	NN	camion / train	Mode of transport
driver	string	NN	non-empty	Driver or operator name
product_id	string	NN	FK → products.product_id	Product transported
departure_warehouse_id	string	NN	FK → warehouses.warehouse_id	Origin warehouse
arrival_warehouse_id	string	NN	FK → warehouses.warehouse_id	Destination warehouse
km_traveled	float	NN	>= 0	Distance in kilometres
trip_date	date	NN	<= today	Date of departure
warehouse_outflow_date	date	NN	>= trip_date	

Column	Type	Null	Constraints	Description
				Date goods left the origin warehouse (stock evolution anchor)
quantity_transported	integer	NN	> 0	Units loaded for transport
quantity_received	integer	—	>= 0, <= quantity_transported	Units actually received at destination
planned_duration_h	float	NN	> 0	Planned trip duration in hours
actual_duration_h	float	—	> 0	Actual trip duration in hours
fuel_consumption_l	float	—	>= 0	Fuel consumed in litres (null for train)
fuel_price_per_l	float	—	>= 0	Fuel price per litre in EUR (null for train)
co2_kg	float	NN	>= 0	CO2 emitted = km_traveled × vehicle_types.co2_per_km_loaded

Foreign keys: - `vehicle_type_id` → `vehicle_types.vehicle_type_id` - `product_id` → `products.product_id` - `departure_warehouse_id` → `warehouses.warehouse_id` - `arrival_warehouse_id` → `warehouses.warehouse_id`

`accounting`

Description: Accounting expense entries — rent, energy, salaries, carbon tax, investments. Links to budget_forecast via cost_center + fiscal_period for budget vs actual analysis.

Column	Type	Null	Constraints	Description
expense_id	string	NN	PK, unique	Unique identifier — prefix `ACC_`
expense_type	string	—	loyer / énergie / salaires / maintenance / logistique / taxe_carbone / investissement / autre	Type of expense
amount	float	NN	>= 0	Amount in EUR
related_entity	string	—	—	ID of the related store, warehouse, etc.
date	date	—	—	Date of the expense
receipt_available	boolean	—	true / false	Whether a receipt exists
cost_center	string	—	magasin / entrepot / siège / logistique	Cost center — joins with budget_forecast
fiscal_period	string	—	YYYY-QN (e.g. 2024-Q1)	Fiscal period — joins with budget_forecast

No foreign keys (related_entity is a generic reference field).

`product_returns`

Description: Products returned by customers with return reason and refund status.

Column	Type	Null	Constraints	Description
return_id	string	NN	PK, unique	Unique identifier — prefix `RET_`
product_id	string	NN	FK → products.product_id	Returned product
customer_id	string	NN	FK → customers.customer_id	Returning customer
return_date	date	NN	<= today	Date of return
reason	string	—	Produit défectueux / Ne correspond pas à la description / Taille incorrecte / Changement d'avis / Livraison en retard / Produit endommagé / Mauvaise référence	Return reason
refunded	boolean	NN	true / false	Whether a refund was issued
quantity	integer	NN	> 0	Number of units returned

Foreign keys: - `product_id` → `products.product_id` - `customer_id` → `customers.customer_id`

`product_reviews`

Description: In-store product reviews collected at POS or by staff. Intentionally overlaps with web reviews in product_catalog_web.json — review_source field distinguishes the two origins and creates cross-source data quality challenges.

Column	Type	Null	Constraints	Description
review_id	string	NN	PK, unique	Unique identifier — prefix `REV_`
product_id	string	NN	FK → products.product_id	Reviewed product
customer_id	string	NN	FK → customers.customer_id	Reviewing customer
rating	float	NN	0 – 10	Product rating
comment	string	—	—	Written comment
review_date	date	—	—	Date of review
store_id	string	—	FK → stores.store_id	Store where review was collected
review_source	string	NN	en magasin / en ligne	Origin of review

Foreign keys: - `product_id` → `products.product_id` - `customer_id` → `customers.customer_id` - `store_id` → `stores.store_id`

`store_maintenance`

Description: Maintenance and renovation interventions in stores.

Column	Type	Null	Constraints	Description
maintenance_id	string	NN	PK, unique	Unique identifier — prefix `MAINT_`
store_id	string	NN	FK → stores.store_id	Concerned store
work_type	string	NN	non-empty	Type of work (e.g. électricité, plomberie, peinture)
date	date	NN	<= today	Date of intervention
amount	float	NN	>= 0	Cost in EUR
contractor	string	—	—	External contractor name

Foreign keys: - `store_id` → `stores.store_id`

`sales_targets`

Description: Sales objectives set per store per period with actual achievement tracking.

Column	Type	Null	Constraints	Description
target_id	string	NN	PK, unique	Unique identifier — prefix `TGT_`
store_id	string	NN	FK → stores.store_id	Concerned store
period	string	NN	YYYY-QN or YYYY-MM	Target period
target_type	string	NN	chiffre_affaires / nombre_transactions / panier_moyen / taux_retour	Type of objective
target_value	float	—	>= 0	Target value
achievement	float	—	>= 0	Actual achieved value

Foreign keys: - `store_id` → `stores.store_id`

`vehicle_types`

Description: Reference table for transport vehicles — mode, size, refrigeration and CO2 emission rates per km. Used by logistics to compute trip-level emissions.

Column	Type	Null	Constraints	Description
vehicle_type_id	string	NN	PK, unique	Unique identifier — prefix `VT_`
transport_mode	string	NN	camion / train	Mode of transport
vehicle_category	string	NN	petit camion / grand camion / semi-remorque / frigorifique petit / frigorifique grand / wagon / train complet	Vehicle category

Column	Type	Null	Constraints	Description
max_capacity_kg	integer	NN	> 0	Maximum load in kilograms
is_refrigerated	boolean	NN	true / false	Whether vehicle has refrigeration
co2_per_km_empty	float	NN	>= 0	CO2 in kg per km when empty
co2_per_km_loaded	float	NN	>= 0	CO2 in kg per km when fully loaded

Static reference table — not subject to dirty data generation.

`carbon_regulations`

Description: Carbon price per country and year (EU ETS or national carbon tax). Joined with logistics.co2_kg and warehouse energy data to compute carbon tax costs flowing into accounting.

Column	Type	Null	Constraints	Description
regulation_id	string	NN	PK, unique	Unique identifier — prefix `REG_`
country	string	NN	France / Allemagne / Espagne / Italie / Portugal	Country
year	integer	NN	>= 2020	Applicable year
co2_price_per_ton	float	NN	>= 0	Carbon price in EUR per metric ton of CO2
regulation_name	string	NN	non-empty	Name of the regulation
applicable_from	date	NN	—	Start date
applicable_to	date	—	—	End date (null = still active)

Static reference table — not subject to dirty data generation.

`store_replenishment`

Description: Replenishment flows from regional warehouses to store-level warehouses. Combined with logistics and transactions, enables full stock evolution calculation at every node.

Column	Type	Null	Constraints	Description
replenishment_id	string	NN	PK, unique	Unique identifier — prefix `REP_`
warehouse_id	string	NN	FK → warehouses.warehouse_id	Source regional warehouse
store_warehouse_id	string	NN	FK → store_warehouses.store_warehouse_id	Destination store warehouse
product_id	string	NN	FK → products.product_id	Replenished product

Column	Type	Null	Constraints	Description
request_date	date	NN	<= today	Date the replenishment was requested
quantity_requested	integer	NN	> 0	Quantity requested
quantity_sent	integer	—	>= 0	Quantity actually sent
status	string	NN	planifié / en_transit / livré / annulé	Replenishment status
shipped_date	date	—	>= request_date	Date goods left the regional warehouse
delivered_date	date	—	>= shipped_date	Date goods arrived at the store warehouse

Foreign keys: - `warehouse_id` → `warehouses.warehouse_id` - `store_warehouse_id` → `store_warehouses.store_warehouse_id` - `product_id` → `products.product_id`

`inventory`

Description: Point-in-time stock snapshots per product per location. Combined with replenishment flows and transaction outflows, enables full stock evolution time series at warehouse and store level.

Column	Type	Null	Constraints	Description
inventory_id	string	NN	PK, unique	Unique identifier — prefix `INV_`
product_id	string	NN	FK → products.product_id	Product
warehouse_id	string	—	FK → warehouses.warehouse_id	Regional warehouse (null if store_warehouse)
store_warehouse_id	string	—	FK → store_warehouses.store_warehouse_id	Store warehouse (null if regional)
snapshot_date	date	NN	<= today	Date of the snapshot
stock_quantity	integer	NN	>= 0	Stock level in units
alert_threshold	integer	NN	>= 0	Minimum threshold before reorder is triggered
expired_quantity	integer	—	>= 0	Units expired since last snapshot

Foreign keys: - `product_id` → `products.product_id` - `warehouse_id` → `warehouses.warehouse_id` - `store_warehouse_id` → `store_warehouses.store_warehouse_id`

`supplier_orders`

Description: Purchase orders sent to suppliers. Enables supply strategy analysis, lead time measurement and supplier reliability benchmarking.

Column	Type	Null	Constraints	Description
order_id	string	NN	PK, unique	Unique identifier — prefix `ORD_`
supplier_id	string	NN	FK → suppliers.supplier_id	Supplier
product_id	string	NN	FK → products.product_id	Ordered product
order_date	date	NN	<= today	Order placement date
expected_delivery_date	date	NN	> order_date	Promised delivery date
actual_delivery_date	date	—	> order_date	Actual delivery date (null if not yet delivered)
quantity	integer	NN	>= suppliers.min_order_qty	Ordered quantity
status	string	NN	en_attente / confirmé / expédié / livré / annulé	Order status
unit_cost	float	NN	>= 0	Negotiated unit cost for this order in EUR

Foreign keys: - `supplier_id` → `suppliers.supplier_id` - `product_id` → `products.product_id`

`price_history`

Description: Historical selling and purchase price per product by period. Enables margin trend and pricing strategy analysis. Intentionally overlaps with price_history array in product_catalog_web.json.

Column	Type	Null	Constraints	Description
price_id	string	NN	PK, unique	Unique identifier — prefix `PRC_`
product_id	string	NN	FK → products.product_id	Product
start_date	date	NN	—	Start of price validity period
end_date	date	—	> start_date	End of price validity (null = current price)
selling_price	float	NN	>= 0	Selling price in EUR
purchase_cost	float	NN	>= 0	Purchase cost in EUR

Foreign keys: - `product_id` → `products.product_id`

`employee_events`

Description: Flat operational HR event log — one row per event per employee. Enables churn rate calculation, salary evolution and career pattern analysis. Complements employee_profiles.json which contains richer qualitative data.

Column	Type	Null	Constraints	Description
event_id	string	NN	PK, unique	Unique identifier — prefix `EVT_`

Column	Type	Null	Constraints	Description
employee_id	string	NN	FK → employees.employee_id	Concerned employee
event_date	date	NN	<= today	Date of the event
event_type	string	NN	embauche / promotion / mutation / augmentation / départ	Type of event
reason	string	—	—	Reason or context
previous_position	string	—	—	Position before event
new_position	string	—	—	Position after event
previous_salary	float	—	>= 0	Salary before event
new_salary	float	—	>= 0	Salary after event

Foreign keys: - `employee_id` → `employees.employee_id`

`economic_zones`

Description: Economic and demographic indicators per country and region. Standalone reference table used for expansion strategy analysis.

Column	Type	Null	Constraints	Description
zone_id	string	NN	PK, unique	Unique identifier — prefix `ZONE_`
country	string	NN	non-empty	Country name
region	string	NN	non-empty	Region or state name
gdp_per_capita	float	NN	> 0	GDP per capita in EUR
consumer_index	float	NN	> 0	Consumer confidence index
population_density	float	NN	> 0	Inhabitants per km²
competitor_count	integer	NN	>= 0	Number of known direct competitors in the zone
market_potential_score	float	NN	0 – 10	Composite market attractiveness score

Static reference table.

`budget_forecast`

Description: Budget forecasts vs actuals per cost center and fiscal period. Joins with accounting on cost_center + fiscal_period for budget variance analysis.

Column	Type	Null	Constraints	Description
forecast_id	string	NN	PK, unique	Unique identifier — prefix `BDG_`
entity_id	string	—	—	ID of the related store or warehouse

Column	Type	Null	Constraints	Description
cost_center	string	NN	magasin / entrepot / siège / logistique	Cost center — join key with accounting
fiscal_period	string	NN	YYYY-QN	Fiscal period — join key with accounting
budget_line	string	NN	loyer / énergie / salaires / maintenance / logistique / taxe_carbone / investissement / autre	Budget line item
planned_amount	float	NN	>= 0	Budgeted amount in EUR
actual_amount	float	—	>= 0	Actual amount (null if period not yet closed)

No foreign keys — joins via shared keys with accounting.

JSON Files

`web_sessions.json`

Description: Customer web sessions — navigation, cart contents and purchase events. Links to transactions when a purchase is completed. Enables online behaviour analysis and delivery delay calculation when crossed with transactions.

Structure:

```
[
  {
    "session_id": "SESS_xxxxxxxx",
    "customer_id": "CUST_xxxxxxxx",
    "connection_time": "2024-03-15T22:14:33",
    "device": "mobile | desktop | tablette",
    "visited_pages": ["accueil", "catalogue", "produit", "panier", "commande"],
    "viewed_products": ["PROD_xxx", "PROD_xxx"],
    "cart": ["PROD_xxx"],
    "purchased": true,
    "transaction_id": "TRX_xxxxxxxx"
  }
]
```

Field notes: - `connection_time` is a full datetime — enables night/weekend web traffic analysis - `transaction_id` is null when `purchased = false`; FK to `transactions.transaction_id` when true - Delivery delay from web: `transactions.actual_delivery_date` – `web_sessions.connection_time` via session_id → transaction_id join chain - `viewed_products`, `cart`, `visited_pages` are arrays requiring UNNEST

`customer_requests.json`

Description: Product demand signals and store feedback from customers. Nested review objects intentionally overlap with product_reviews.csv — same customer may have different ratings in both sources (web vs in-store).

Structure:

```
[
  {
    "customer_id": "CUST_xxxxxxxx",
    "request_date": "2024-06-01",
    "priority": "faible | moyenne | élevée",
    "requested_products": ["PROD_xxx", "PROD_xxx"],
    "reviews": [
      {
        "store_id": "MAG_xxxxxxxx",
        "rating": 7.5,
        "comment": "Commentaire libre",
        "source": "en ligne"
      }
    ]
  }
]
```

Field notes: - `reviews[].source` is always `'en ligne'` here vs `'en magasin'` in product_reviews.csv - `request_date` bounded to company operating period (>= 2015-01-01)

`marketing_notifications.json`

Description: Notification history per customer per campaign. Each entry contains an array of send attempts with per-device delivery metadata — multi-level unnesting required.

Structure:

```
[
  {
    "campaign_id": "CAMP_xxxxxxxx",
    "customer_id": "CUST_xxxxxxxx",
    "sends": [
      {
        "channel": "email | sms",
        "sent_at": "2024-11-02T09:45:00",
        "device": "mobile | desktop | tablette",
        "delivery_status": "livré | échoué | en_attente",
        "opened": true,
        "clicked": false,
        "unsubscribed": false
      }
    ]
  }
]
```

Field notes: - `campaign_id` FK to `marketing.campaign_group_id` - One customer can receive multiple sends within the same campaign (A/B testing, retry on failure) - `sends` array is the unnesting challenge — each send has independent delivery metadata

---

`stock_alerts.json`

Description: Stock alert state per warehouse with per-product alert history. Three nesting levels: warehouse → products → quantity history timeline. Complements inventory.csv snapshots.

Structure:

```
[
  {
    "warehouse_id": "ENT_xxxxxxxx",
    "snapshot_date": "2024-10-01",
    "products": [
      {
        "product_id": "PROD_xxxxxxxx",
        "current_qty": 120,
        "alert_threshold": 200,
        "alert_level": "critique | warning | normal",
        "history": [
          { "date": "2024-09-01", "qty": 350 },
          { "date": "2024-09-15", "qty": 220 },
          { "date": "2024-10-01", "qty": 120 }
        ]
      }
    ]
  }
]
```

Field notes: - `alert_level` derivation: `critique` if current_qty < 0.5 × threshold, `warning` if < threshold, `normal` otherwise - Students must reconcile this with inventory.csv snapshots — values may differ slightly

---

`employee_profiles.json`

Description: Rich employee career profiles — skills, training history, performance reviews and certifications. Complements employee_events.csv (operational flat events) with qualitative and nested data that does not flatten naturally.

Structure:

```
[
  {
    "employee_id": "EMP_xxxxxxxx",
    "skills": ["compétence1", "compétence2"],
    "certifications": ["certification1"],
    "trainings": [
      {
        "date": "2023-03-10",
        "title": "Intitulé de la formation",
        "duration_days": 2,
        "score": 8.5
      }
    ],
    "performance_reviews": [
      {
        "period": "2023-Q4",
        "score": 7.0,
        "comment": "Commentaire manager",
        "reviewer_id": "EMP_xxxxxxxx"
      }
    ]
  }
]
```

Field notes: - `employee_id` FK to `employees.employee_id` - `performance_reviews[].reviewer_id` FK to `employees.employee_id` (the reviewing manager) - Cross-analysis with employee_events.csv enables churn root cause analysis

---

`product_catalog_web.json`

Description: Web product catalogue with descriptions, tags, customer web reviews and price history. Three nesting levels. Web reviews overlap with product_reviews.csv for cross-source data quality exercises.

Structure:

```
[
  {
    "product_id": "PROD_xxxxxxxx",
    "description": "Description longue du produit en français",
    "tags": ["tag1", "tag2"],
    "web_reviews": [
      {
        "customer_id": "CUST_xxxxxxxx",
        "session_id": "SESS_xxxxxxxx",
        "rating": 8.0,
        "comment": "Commentaire en ligne",
        "date": "2024-07-15"
      }
    ],
    "price_history": [
      { "date": "2023-01-01", "price": 49.99 },
      { "date": "2024-01-01", "price": 54.99 }
    ]
  }
]
```

Field notes: - `web_reviews[].customer_id` FK to `customers.customer_id` - `web_reviews[].session_id` FK to `web_sessions.session_id` - `price_history` overlaps with price_history.csv — minor value differences possible (intentional dirty data) - `tags` enables product segmentation analysis

---

Supply Chain Flow

```
suppliers
    │
    │ supplier_orders (purchase orders + delivery dates)
    ▼
warehouses (regional DCs)
    │
    │ logistics (truck/train trips — km, CO2, delays)
    │ store_replenishment (warehouse → store backroom flows)
    ▼
store_warehouses (store backrooms)
    │
    │ (implicit: daily shelf replenishment)
    ▼
stores (sales floor) ◄── transactions (customer purchases)
```

Stock evolution at any node: - Warehouse inflow: `supplier_orders.actual_delivery_date` - Warehouse outflow: `logistics.warehouse_outflow_date` - Store warehouse inflow: `store_replenishment.delivered_date` - Store warehouse outflow: `transactions.purchase_date` (aggregated by product + store) - Snapshots: `inventory.snapshot_date`

Carbon cost calculation:

```
logistics.co2_kg × carbon_regulations.co2_price_per_ton / 1000
→ accounting entry with expense_type = 'taxe_carbone'
```